

Ripple: An Open, AI-Formalized Lean 4 Framework for Computing with CRNs

Ho-Lin Chen^{*1} and Xiang Huang^{†2}

¹Department of Electrical Engineering, National Taiwan University, Taiwan,
holinchen@ntu.edu.tw ORCID 0000-0002-6171-9962

²Department of Computer Science, University of Illinois Springfield, USA, xhuan5@uis.edu
ORCID 0000-0002-4815-6130

Abstract

We present **Ripple**, an open, AI-formalized Lean 4 framework for the mathematics of computing real numbers with chemical reaction networks (CRNs). Ripple formalizes the full ladder of models — the GPAC / CRN continuum and the CRN-computable reals, the large-population-protocol (LPP) compilation pipeline, and a continuous-time Markov chain (CTMC) layer bridged to the deterministic mean-field limit by three machine-checked versions of Kurtz’s theorem, and two Turing-completeness results — the Bournez–Graça–Pouly GPAC Turing-completeness construction and the Soloveichik–Cook–Winfree–Bruck stochastic-CRN universality theorem. The development is reliable (its core constructions are verified to depend on exactly the three Mathlib foundational axioms, with no `sorry`); it exposed genuine, fixable gaps in published proofs (the approximate-majority convergence argument and the LPP main theorem); and it proves new results — a fully machine-checked construction of Apéry’s constant $\zeta(3)$ as a CRN-computable number via its holonomic generating function, the same recipe turning the modular $1/\pi$ series of Ramanujan into a sharp open problem. The formalization was carried out predominantly by AI agents using only publicly available models, so the workflow is reproducible.

Keywords: formal verification, Lean 4, chemical reaction networks, analog computation, GPAC, computable real numbers, population protocols, Kurtz’s mean-field theorem.

1 Introduction

Interactive theorem provers have lately moved from a curiosity at the edge of mathematics to a working tool inside it. Lean 4 and its library Mathlib now carry machine-checked proofs once thought out of reach — the Liquid Tensor Experiment [20] and the polynomial Freiman–Ruzsa conjecture [11] — and AI agents have begun proving theorems autonomously, settling open Erdős and OEIS problems [24] and disproving the Erdős unit-distance conjecture [18]. Those breakthroughs used proprietary internal models; the work here was done entirely with *publicly available* ones (Claude Opus 4.6–4.8, Claude Fable 5, GPT-5.4–5.6), so the AI-assisted workflow is reproducible by anyone.

We bring this to molecular programming. **Ripple** (<https://github.com/zinan-huang/Ripple>) is an *open, AI-formalized* Lean 4 framework for the mathematics of computing with chemical reaction

^{*}Supported in part by NSTC (Taiwan) grant 113-2221-E-002-204-MY3.

[†]Supported in part by Department of Energy EXPRESS grant DE-SC0024278.

networks, covering the theory of CRN-computable real numbers, the population-protocol majority algorithms, the Turing-completeness of both the deterministic (GPAC/BGP) and stochastic (SCWB) models, and the supporting mathematical infrastructure — continuous-time Markov chains, Kurtz’s mean-field theorem, Frobenius regular-singular theory, and concentration inequalities. These are exact, layered, and spread across many papers — the kind of mathematics where formalization earns its place by catching gaps that informal proofs leave behind.

The formalized theory. As an initial thread Ripple formalizes the theory of *CRN-computable real numbers*, building on three prior papers [14, 13, 9] and covering the full ladder of models:

- **Continuous / deterministic.** The GPAC / polynomial-IVP form, CRN mass-action dynamics, the CRN-computable reals, dual-rail compilation, and the large-population-protocol (LPP) refinement, under one shared Lean definition of “CRN computes α ”.
- **Stochastic / discrete.** The Angluin–Aspnes–Eisenstat 3-state *approximate majority* protocol with its full $O(n \log n)$ convergence proof [2], and the deterministic correctness chain of the Doty et al. optimal *exact majority* protocol [10].
- **Turing completeness.** The Bournez–Graça–Pouly (BGP) construction [6] showing that polynomial IVPs can simulate arbitrary Turing machines (≈ 170 Lean files), and the Soloveichik–Cook–Winfree–Bruck stochastic CRN universality theorem [23] — the encoding of Turing machines into stochastic CRNs (≈ 90 Lean files).
- **Kurtz’s theorem.** The mean-field limit theorem linking the two regimes, making precise how the deterministic ODE is the $N \rightarrow \infty$ limit of the stochastic CRN [16].

Four things formalization buys. Formalizing this theory does four things informal mathematics cannot.

1. **It is reliable.** A machine-checked proof leaves no step for a reader to fill in: every inference is checked by Lean’s small trusted kernel, so trust reduces to that kernel plus three foundational axioms. We verified, via Lean’s axiom-dependency check, that the core constructions ($\pi/4$, $\zeta(3)$, Catalan, LPP, Kurtz) depend on exactly those and nothing more — no **sorry**, no hidden hypotheses (§10).
2. **It exposes gaps.** The formalization uncovered genuine, fixable gaps in published constructions — in the original approximate-majority convergence proof and in the LPP main theorem among them — each caught as a Lean compile error or explicit counterexample (§6).
3. **It is open and extensible.** Adding a computable number is a plug-in; more broadly the foundation — the GPAC core, the CTMC/Kurtz layer, the compilation pipelines — is a base others can build new threads on (§2).
4. **It proves new things — and finds new problems.** Ripple gives a fully machine-checked construction of $\zeta(3)$ as a CRN-computable number via its holonomic generating function (§7); the same recipe applied to the modular $1/\pi$ series of Ramanujan surfaces a sharp open problem, its series anchor being a fixed point of the drive (§8).

Organization. §2 is an overview of *what the Lean development contains* — a map of the framework and the threads it carries. §3 then develops the model ladder in full, from the GPAC / CRN continuum, through the foundational constants and the LPP compilation pipeline, to the CTMC layer, Kurtz’s theorem, and the LPP→Kurtz bridge from finite-horizon convergence to readout correctness. §6 records the gaps the formalization exposed; §7 and §8 then give the new $\zeta(3)$ and π constructions in detail, with both the informal arguments and the shape of the Lean proofs. Finally §9 reports how the formalization was actually done, and §10 delimits the trust footprint.

2 An Open Framework: Foundations, Threads, and Extension

Ripple is built to be *open*: a reusable foundation that others can extend, or borrow from to build their own developments. It is best read not as a closed result but as an *initiative* — an attempt that carries multiple threads of research forward, on infrastructure deliberately made general. Two layers make this concrete. Figure 1 is a map of the repository — each pillar with its size and what it formalizes — showing both the reusable foundation and the threads (completed and in progress) built on it.

Foundational infrastructure (reusable, and new to Lean). A large part of Ripple is general-purpose and independent of any particular target number. It includes the GPAC/PIVP core and the single `CertifiedBoundedTimeComputable` type (§3) that pins down what “computes α ” means; the dual-rail compiler; the four-stage LPP compilation pipeline; and—built from the ground up—a continuous-time Markov chain layer (DTMC, CTMC, the density process) together with three machine-checked versions of *Kurtz’s mean-field theorem*. These are foundational definitions and theorems: a verified CTMC theory and a formal Kurtz limit did not previously exist in Mathlib, and they are reusable far beyond this paper — by anyone formalizing stochastic chemical kinetics, population dynamics, or mean-field limits, whether or not they care about computable numbers.

Built to extend. Because the CRN-computable class is a Lean type, adding a new computable number is a plug-in: supply a PIVP, prove boundedness and an exponential modulus, invoke the pipeline; a definition proved once is reused everywhere, the type system guaranteeing the meaning never silently drifts between theorems written years apart. More broadly, the framework is open in the strong sense: the foundational layer (the GPAC core, the CTMC/Kurtz theory, the compilation pipelines) is a base on which others can mount entirely new threads of research, or which they can take piecemeal for their own.

3 The Model Ladder

This section develops the model ladder sketched in the introduction into a self-contained account of what Ripple formalizes and how the pieces connect: the analog model (§3.1), CRN-computability (§3.2), the foundational constants and Catalan (§3.3), the LPP compilation pipeline (§3.4), the stochastic CTMC layer with Kurtz’s mean-field theorem (§3.5), and the LPP→Kurtz connection (§3.6).

Throughout: a CRN in its mass-action limit is a polynomial ODE system $\dot{x}_i = p_i(x) - x_i d_i(x)$ with p_i, d_i having non-negative coefficients; dropping the sign structure gives a PIVP $\dot{y} = p(y)$; and “ α is CRN-computable” means a coordinate satisfies $|y_k(t) - \alpha| \leq Ce^{-t}$.

Ripple/ pillar	files / lines / defs	what it formalizes
Core/	15 / 7.2k / 152	GPAC/PIVP, bounded-time complexity, CRN pipeline
ODE/	1 / 0.3k / 5	Lyapunov / scalar convergence barriers
DualRail/	18 / 14k / 358	dual-rail encoding of polynomial dynamics
LPP/	43 / 95k / 2.1k	large-population protocols [13]; algebraic construction; QBee pipeline; compilation stages
Number/	21 / 14k / 565	$e, \pi, \ln 2, \gamma$, Catalan G , Dottie, $\zeta(3)$ (Fermi/Apéry)
Number/Frobenius/	23 / 57k / 2.1k	Frobenius regular-singular theory; Apéry conifold
Number/Modular/	30 / 27k / 1.3k	modular forms, Φ_{41} , CM-163, $j(\tau_{163})$
Number/Hypergeom./	3 / 5k / 173	Clausen, Picard–Fuchs Wronskian, Chowla–Selberg; Ramanujan reduction
CTMC/	11 / 35k / 1.2k	DTMC/CTMC, density process, Doob bound, absorbing states
Kurtz/	9 / 3.6k / 78	Kurtz’s mean-field theorem (three versions); log-horizon tube
Probability/	3 / 1.1k / 28	Bennett exponential-moment lemma; discrete Freedman inequality
sCRNUniv./	92 / 45k / 3.4k	stochastic CRN Turing completeness [23]
BoundedUniv./	214 / 161k / 3.6k	GPAC Turing completeness [6]; bounded extensions (in preparation)
Analysis/	1 / 0.3k / 5	stable Grönwall lemma
<i>population-protocol formalizations (majority)</i>		
PopulationProtocol/	435 / 303k / 8.3k	<i>exact majority</i> (Doty et al.); <i>approximate majority</i> [2] (drift gap found and fixed); <i>self-stabilizing</i> [15]
Total	≈ 920 / $\approx 768k$ / $\approx 23.4k$	

Figure 1: Map of the Ripple framework. “defs” counts top-level **theorem/lemma/def** declarations. The upper block is the core development; the lower block groups the population-protocol majority formalizations. Notable additions since submission: **Probability/** (Bennett/Freedman), **sCRNUniv./** (the Soloveichik–Cook–Winfree–Bruck stochastic CRN universality theorem), **BoundedUniv./** (the Bournez–Graça–Pouly GPAC Turing completeness construction, plus bounded extensions in preparation), and a substantially expanded **PopulationProtocol/** pillar now incorporating all three majority formalizations (Doty et al., Angluin–Aspnes–Eisenstat with the drift-gap fix of §6(A), and Kanaya et al.).

3.1 The analog model: GPAC, PIVP, and the mass-action CRN

The continuous model underneath everything is Shannon’s *General Purpose Analog Computer* (GPAC), a circuit of constant, adder, multiplier, and integrator elements. Shannon proved a GPAC computes exactly the *differentially algebraic* functions; equivalently, the components of a solution to a *polynomial initial value problem* (PIVP)

$$\dot{y} = p(y), \quad y(0) = y_0 \in \mathbb{Q}^d, \quad p \in \mathbb{Q}[y_1, \dots, y_d]^d.$$

Long believed strictly weaker than Turing computation (the Gamma function is not differentially algebraic), the GPAC was shown by Bournez, Campagnolo, Graça, and Hainry (2007) to compute all computable functions on computable compact intervals, polynomial-time equivalent to the Turing machine: the polynomial vector field is universal.

A chemical reaction network is the same mathematics with “concentration” for “voltage”: a reaction $A + B \rightarrow C + D$ under mass-action kinetics fires at a rate proportional to the product of reactant concentrations, so each species’ rate of change is a polynomial. The mass-action CRN is a

PIVP wearing a sign constraint — production terms are non-negative, and every negative monomial in $\dot{x}_i$ must carry x_i as a factor (the Hárs–Tóth “lack of negative cross-effects” condition). Hárs and Tóth (1981) proved this condition is exactly what makes a polynomial ODE realizable by a mass-action CRN, with an explicit one-reaction-per-monomial mechanism. For Ripple only the easy direction matters: every bounded PIVP below splits into production p_i and degradation d_i with the right signs, which is what Lean’s `PolyCRNDecomposition` structure records.

3.2 CRN-computability

A real number α is **CRN-computable** if some PIVP with rational data has a coordinate $y_k(t)$ that stays bounded and satisfies

$$|y_k(t) - \alpha| \leq C e^{-t} \quad \text{for all } t \geq 0.$$

This is the single notion of computability used throughout: a bounded polynomial system whose output coordinate converges to the target with a certified exponential modulus.

The source papers use “bounded CRN computes α ” in subtly different informal senses; in Lean there is exactly one carrier,

$$\text{CertifiedBoundedTimeComputable } (d : \mathbb{N}) (\alpha : \mathbb{R})$$

(`Ripple/Core/BoundedTime.lean`), bundling a PIVP of dimension d , a designated output coordinate, a boundedness witness, and an explicit exponential-convergence modulus. Every theorem in the project — constant constructions, dual-rail compiler, LPP pipeline — consumes or produces this one structure, so “computes α ” is type-checked to be identical across results written years apart. A forgetful coercion drops the certificate when only the weaker existence statement (`BoundedTimeComputable`) is wanted.

3.3 Foundational constants and Catalan’s constant G

Ripple re-formalizes the elementary constants — e , π (through $\pi/4 = \arctan 1$), $\ln 2$, $\frac{1}{2}e^{-1}$, the Euler–Mascheroni γ (through $\gamma = 1 - \Gamma'(2)$), and the Dottie number — each as a bounded PIVP with an explicit exponential modulus, all axiom-free [14]; the only reciprocal any needs is the bounded state $1/(1+t)$ (satisfying $r' = -r^2$), which keeps them inside the polynomial-PIVP class. The worked example of the integral-to-PIVP recipe is Catalan’s constant $G = \sum_{n \geq 0} (-1)^n / (2n+1)^2 \approx 0.9160$, with the Laplace-type representation $G = \int_0^\infty \frac{s e^{-s}}{1+e^{-2s}} ds$. Its only non-polynomial factor is $1/(1+e^{-2s})$; carrying it as a state $W := 1/(1+e^{-2t})$, which satisfies a quadratic ODE ([13, Cor. 19]), turns the whole integral into the four-state polynomial PIVP

$$\begin{cases} E' = -E, \\ R' = E - R, \\ W' = 2 E^2 W^2, \\ G' = R \cdot W, \end{cases} \quad (E, R, W, G)(0) = (1, 0, \tfrac{1}{2}, 0),$$

with closed forms $E = e^{-t}$, $R = t e^{-t}$, $W = 1/(1+e^{-2t})$, so $G(t) \rightarrow G$ with modulus $|G(t) - G| \leq (t+1)e^{-t}$. This is the theorem `catalan_is_lpp_computable` (`Ripple/Number/CatalanCertified.lean`), verified clean on the same generic bounded-PIVP-plus-modulus machinery every target uses.

3.4 The LPP compilation pipeline

The *large-population protocol* (LPP) model is the CRN model with bounded populations: instead of arbitrary real concentrations, one tracks fractions $x_i = n_i/N$ of a fixed population of size N . The central result is `bounded_crn_is_lpp_computable_unconditional` (`Ripple/LPP/BoundedLPP.lean`), verified clean: *every* bounded certified PIVP compiles into an LPP computing the same number. It is strictly stronger than [13], which implicitly assumed the certified computation stays inside $[0, 1]$ at all times — a hypothesis the formalization showed can fail, since the output may transiently overshoot while converging from above (see §6(B)). The repair is a saturating low-pass filter $y' = (x - y)(U - y)$ with $U \in (\alpha, 1) \cap \mathbb{Q}$, which clamps the output into a user-chosen interval without changing the limit.

The compiler is a four-stage pipeline, each stage a separately verified transformation:

- (i) **Quadraticization.** Reduce an arbitrary-degree polynomial vector field to degree ≤ 2 via auxiliary product variables $z = x_i x_j$; preserves the solution and modulus, adds coordinates.
- (ii) **Bound-to-small- λ closure.** Rescale time/rate so all reaction rates fit under the LPP budget (“small- λ ”), using the boundedness certificate to control the rescaling.
- (iii) **Triple-product discretization.** Realize the continuous quadratic dynamics by population-level interactions, matching mass-action products to the bimolecular rates of a population of size N .
- (iv) **Pure-LPP embedding.** Embed the discretized system into the formal LPP model, producing the `IsLPPComputable` witness.

A corollary: every algebraic $\alpha \in [0, 1]$ is LPP-computable, via the quadratic $\dot{x}_1 = \varepsilon(a_0 + a_1 x_1 + a_2 x_1^2)$ with the target root as a stable fixed point. The formalization also repaired a gap in the Bournez–Fraigniaud–Koeqler conservation trick (another instance of §6): the balancing variable $\dot{x}_\delta = -\sum_j \dot{x}_j$ is not CRN-implementable as written, and Ripple’s balancing-dilation repair recovers it.

3.5 The CTMC layer and Kurtz’s mean-field theorem

A real CRN with N molecules is a stochastic CTMC; the deterministic ODE is the $N \rightarrow \infty$ limit (Kurtz, 1970). Ripple’s `CTMC/` and `Kurtz/` layers formalize this `sorry-free`. The density process $\bar{X}^N(t) = X(t)/N$ admits the martingale decomposition

$$\bar{X}^N(t) = \bar{X}^N(0) + \int_0^t F(\bar{X}^N(s)) ds + M^N(t), \quad \mathbb{E}[\sup_{s \leq t} \|M^N(s)\|^2] \leq Ct/N.$$

Three `sorry-free` versions of the limit are proved: weak (convergence in probability, $\sup_{t \leq T} \|\bar{X}^N - x\| \rightarrow 0$), strong (a.s. rate $O(\log N/\sqrt{N})$ via Azuma–Hoeffding + Borel–Cantelli), and CLT-scale ($\mathbb{E}[\sup \|\bar{X}^N - x\|^2] \leq C/N$).

3.6 The LPP→Kurtz connection: from finite-horizon convergence to readout correctness

An LPP computes α through **two limits in a definite order**:

$$\underbrace{\text{stochastic PP at size } N}_{\text{finite, random}} \xrightarrow[N \rightarrow \infty]{\text{Kurtz}} \underbrace{\text{mean-field ODE } x(t)}_{\text{deterministic}} \xrightarrow[t \rightarrow \infty]{} \underbrace{\alpha}_{\text{the target}}.$$

Kurtz gives the first arrow on *compact* intervals; it says nothing as $T \rightarrow \infty$. Passing to a genuine readout guarantee requires two additional hypotheses: (1) exponential stability of the limit ODE (the CRN-computability certificate), and (2) uniform-in- N late-time concentration (a Foster–Lyapunov bound forbidding late escape). Ripple establishes (1) and the compact-interval Kurtz step **sorry-free**; hypothesis (2) is not discharged but bracketed as an explicit modeling assumption via the **Computes** definition, which is stated on the post- $N \rightarrow \infty$ ODE. The order-of-limits content is encoded structurally — **Computes** is defined on the *post*-population-limit ODE, so the file factors the claim into these three links rather than asserting a monolithic $\lim_t \lim_N = \lim_N \lim_t$. Of the two hypotheses above, Ripple discharges (1); hypothesis (2), the uniform-in- N late-time concentration, it does *not*, and **Computes** is stated on the post- $N \rightarrow \infty$ ODE precisely so this bound is bracketed as an explicit modeling assumption rather than silently asserted.

A companion result (**Ripple/Kurtz/MeanFieldLogHorizon.lean**) takes a first quantitative step past this barrier: composing a stable-Grönwall shadowing bound, a Freedman martingale tail, and a lifted-stability certificate, it proves a tube estimate on the *growing* horizon $[0, \log N/(2\eta)]$ — $\mathbb{P}(\sup_{t \leq \log N/(2\eta)} \|\bar{X}^N(t) - \psi(t)\|_\infty > C_K \log N/\sqrt{N}) \leq N^{-p}$ — with a Lipschitz-readout corollary bounding $|R(\bar{X}^N(T_N)) - \nu|$ at the endpoint. It is stated modulo those three named inputs rather than fully unconditionally, but it is precisely the finite- N readout guarantee the compact-interval theorem does not by itself provide.

The upshot. The deterministic constructions (§3.3) and the LPP pipeline (§3.4) live entirely on the ODE side; the CTMC/Kurtz layer (§3.5) connects that side back to the *physical* stochastic CRN with finitely many molecules. This subsection is the seam: the formal statement that the number an LPP computes — defined through finite- N stochastic dynamics — is exactly the number the corresponding PIVP converges to, with the order of the population and time limits controlled. Without it, “the LPP computes α ” and “the PIVP converges to α ” would be two unrelated assertions; with it, they are the same assertion, type-checked.

4 Population-Protocol Majority Formalizations

Alongside the CRN-computability thread, Ripple formalizes three landmark majority protocols from the population-protocol literature — approximate, exact, and self-stabilizing exact majority — together making up the largest pillar of the repository (≈ 435 files, $\approx 303\text{k}$ lines; Figure 1).

Approximate majority (Angluin–Aspnes–Eisenstat, 2008). The classic 3-state protocol [2]: blanks adopt the opinion they meet, opposing opinions cancel into a blank, and with high probability the population reaches consensus on the initial majority in $O(n \log n)$ interactions. Ripple formalizes the *full probabilistic convergence theorem* — not just stable correctness but the $O(n \log n)$ high-probability time bound — in $\approx 6,650$ lines of Lean, zero **sorry**. This is the formalization in which Gap A was found: the central-region multiplicative drift inequality of the original proof sketch is false, with an explicit $n = 4$ counterexample, and the Lean proof carries a corrected argument (§6).

Exact majority (Doty et al., 2021). The time- and space-optimal *exact* majority protocol [10], which decides the majority correctly with certainty using $O(\log n)$ states per agent. Ripple formalizes the deterministic correctness chain — the protocol’s phase structure, its invariants, and the stable-correctness main theorem — along with the $O(\log n)$ state-count bound. At $\approx 184\text{k}$ lines

across ≈ 300 files it is the largest single formalization in the repository, reflecting the intricacy of the protocol’s cancellation, doubling/halving, and clock phases.

Self-stabilizing exact majority (Kanaya et al., 2025). The time- and space-optimal *silent self-stabilizing* exact majority protocol [15], which must reach a correct, unchanging consensus from an *arbitrary* initial configuration. Ripple formalizes the paper’s four theorems: the impossibility of self-stabilizing majority without knowledge of n (`impossibility_without_n`), the space lower bound (`space_lower_bound`), convergence, and the time upper bound. The protocol composes with the OPTIMAL-SILENT-SSR ranking subprotocol of Burman, Chen, Chen, Doty, Nowak, Severson, and Xu [7], whose deterministic convergence proof Ripple formalizes in full (≈ 16 k lines in `BurmanConvergenceFinal.lean`): the ranking field construction, the phase-2 propagation/reset, and the epidemic-timer branch to consensus are all machine-checked. The top-level theorem for the concrete composed protocol (`P_EM_solves_SSEM_final`) is unconditional — no external hypotheses — for all $n \geq 4$. The development is ≈ 111 k lines across ≈ 99 files.

5 Turing-Completeness Formalizations

Ripple formalizes two classical universality results, one on each side of the model ladder: the deterministic GPAC/polynomial-IVP construction of Bournez–Graça–Pouly, and the stochastic-CRN construction of Soloveichik–Cook–Winfree–Bruck. Both show that a molecular-computation model is Turing-complete, and both are machine-checked with the standard three-axiom footprint.

Deterministic: polynomial IVPs simulate Turing machines (BGP, 2017). Bournez, Graça, and Pouly proved that a polynomial initial-value problem — equivalently, a GPAC — can simulate an arbitrary Turing machine, so the class of polynomial ODEs is Turing-complete [6]. Ripple formalizes the BGP construction in ≈ 173 Lean files under `BoundedUniversality/BGP/`: the smooth encoding of a machine configuration into real coordinates, the rational periodic clock, the single-step transition map, and their composition into one polynomial vector field whose trajectory tracks the machine’s computation. The headline theorem `bounded_pivp_turing_complete` states that a single rational bounded PIVP eventually-threshold-simulates an undecidable machine; its `#print axioms` footprint is exactly `propext`, `Classical.choice`, `Quot.sound` and nothing more (“clean-3”), with no `sorry` and no custom axiom. The bounded-analog-complexity extensions built on this construction are in preparation.

Stochastic: Turing machines encode into stochastic CRNs (SCWB, 2008). Soloveichik, Cook, Winfree, and Bruck showed that finite stochastic chemical reaction networks are Turing-universal: any Turing machine can be encoded into a stochastic CRN that simulates it with arbitrarily small error probability [23]. Ripple formalizes this in ≈ 92 Lean files under `sCRNUniversality/`, covering three layers: (i) the deterministic Turing-machine-to-CRN encoding, in which a four-phase bimolecular refinement network reproduces each machine step as a reachability fact (`statePairTransfer_crn_reaches_of_ctm_steps`); (ii) the mass-action stochastic bridge, which constructs the path-space probability measure from the mass-action step kernels via Mathlib’s Ionescu–Tulcea theorem and bounds the per-step error probability; and (iii) the decidability of Petri-net coverability, obtained through the backward coverability algorithm with termination proved from a well-quasi-order argument (Dickson’s lemma). The paper-facing theorems are machine-checked with the same three-axiom footprint.

6 Gaps Exposed by Formalization

A machine-checked proof has no human reader to fill in the routine steps. Every logical step must be justified, every case covered, every interaction pattern analyzed. In the course of formalizing the constructions that make up the Ripple pipeline — both new constructions and the published results they build on — this discipline has surfaced a number of genuine gaps. In each case the published *theorem* is correct; the gap lives in a proof sketch, an intermediate construction, or an unstated hypothesis, and surfaces in Lean as a compile error or as an explicit counterexample rather than as a footnote.

We document three representative gaps in detail below — the mathematics of the original claim, the precise reason it fails or is incomplete, the explicit counterexample where one exists, and the fix the formalization carries — and note that the formalization turned up others beyond these (for instance an interior-saddle counterexample to a 2024 algebraic-CRN basin-entry step, and an atomic-initialization subtlety in the exact-majority error-jump paths).

The three gaps detailed here.

- (A) §6.1 — Angluin–Aspnes–Eisenstat (2008) approximate majority: the central-region multiplicative drift inequality is *false*; explicit $n = 4$ counterexample; product-form supermartingale fix. (The final $O(n \log n)$ theorem is correct.)
- (B) §6.2 — the large-population-protocol [13] main theorem: the certified CRN output can transiently leave $[0, 1]$; saturating low-pass filter fix.
- (C) §6.3 — reaching algebraic numbers of *arbitrary degree* in the urn model needs the non-existence of solutions to the Catalan equation $x^p - y^q = 1$ (Catalan’s conjecture); the classical Cassels (1960) descent supplies only *divisibility*, pinning down exactly which case the argument must still close.

6.1 (A) The approximate-majority central-region drift inequality is false

This is the headline gap. The 3-state approximate majority protocol of Angluin, Aspnes, and Eisenstat (*Distributed Computing*, 2008) is one of the most studied population protocols, and its $O(n \log n)$ convergence proof is a classic. Formalizing that proof in Lean 4 (the **pp-proof** development, $\approx 6,650$ lines, zero **sorry**, now part of the **PopulationProtocol1/** pillar of Figure 1) revealed that one intermediate step in the original proof sketch does not hold as stated. The final theorem is correct; the central region needs a different argument from the one the sketch suggests.

The protocol. There are n agents, each in one of three states: x (opinion A), y (opinion B), or b (blank/undecided). At each step two agents are chosen uniformly at random and updated by

$$xb \rightarrow xx, \quad yb \rightarrow yy, \quad xy \rightarrow xb \text{ or } xy \rightarrow yb.$$

Blanks adopt the opinion they meet; opposing opinions cancel into a blank. The theorem is that the protocol reaches consensus in $O(n \log n)$ interactions with high probability, matching the information-theoretic lower bound.

The potential and the four-region partition. The proof uses the potential function

$$f = u^2 + 2n, \quad u = x - y \quad (\text{the opinion gap}).$$

The configuration space is partitioned into four regions by which count is large (threshold $7n/8$):

$$\text{Large-}x \ (x \geq \tfrac{7n}{8}), \quad \text{Large-}y \ (y \geq \tfrac{7n}{8}), \quad \text{Large-}b \ (b \geq \tfrac{7n}{8}), \quad \text{Central} \ (\text{no count} \geq \tfrac{7n}{8}).$$

For the three *corner* regions the proof establishes a multiplicative drift: the expected value of $1/f$ contracts by a factor $(1 - \delta)$ per step, $\delta = \Theta(1/n)$, which gives geometric decay by standard arguments.

The suggested inequality. The paper's Lemma 4 concerns the central region. The surrounding proof sketch implicitly carries the same multiplicative-drift framework into the central region, i.e. it needs, for $f' = f(C_{t+1})$ the potential after one step,

$$\mathbb{E}\left[\frac{1}{f'} \mid \mathcal{F}_t\right] \leq (1 - \delta) \frac{1}{f_t} \quad (\text{central region}). \quad (1)$$

Inequality (1) does not hold in general.

The counterexample ($n = 4$). Take $n = 4$, with counts $x = 1, b = 0, y = 3$. This is a central configuration: the threshold is $7 \cdot 4/8 = 3.5$, and the largest count is $y = 3 < 3.5$. The opinion gap is $u = x - y = -2$, so the potential is

$$f = (1 - 3)^2 + 2 \cdot 4 = 4 + 8 = 12.$$

There are three outcomes: a null yy draw (probability $\frac{1}{2}$, $f' = 12$); $xy \rightarrow xb$ (probability $\frac{1}{4}$, giving $(1, 1, 2)$ with $u = -1$, $f' = 9$); and $xy \rightarrow yb$ (probability $\frac{1}{4}$, giving $(0, 1, 3)$ with $u = -3$, $f' = 17$). Hence $\mathbb{E}[1/f'] = \frac{1}{2} \cdot \frac{1}{12} + \frac{1}{4} \cdot \frac{1}{9} + \frac{1}{4} \cdot \frac{1}{17} = \frac{103}{1224}$, so

$$\frac{\mathbb{E}[1/f']}{1/f} = \frac{103}{102} > 1. \quad (2)$$

So $1/f$ *increases* on average from this configuration — the opposite of the contraction (1) requires (which would need the ratio $\leq 1 - \delta < 1$). Truncating to the active central region does not rescue the estimate: *all* transitions out of $(1, 0, 3)$ remain in the central region, so there is no absorption boundary to provide the missing correction. The counterexample is not an artifact of small n : it was verified computationally to persist for every n from 4 through 50.

The reason it fails: Jensen runs the wrong way. The function $x \mapsto 1/x$ is *convex* on $(0, \infty)$, so Jensen's inequality gives

$$\mathbb{E}\left[\frac{1}{f'}\right] \geq \frac{1}{\mathbb{E}[f']}, \quad (3)$$

with a gap between the two sides of second order — of size $\text{Var}(f')/\mathbb{E}[f']^3$ up to constants. A favorable mean drift $\mathbb{E}[\Delta f] \geq 0$ therefore controls only the *right-hand* side of (3); the quantity that (1) needs to contract is the left-hand side, which sits *above* it by the convexity correction. The inequality one wants is fighting the convexity of $1/x$. In the corner regions the step size of f is small relative to f itself, so the convexity (second order) error is negligible and the first-order drift wins. In the central region this is no longer true: a single xy interaction changes u by ± 1 , moving f by an amount of order

$$|\Delta f| \sim |(u \pm 1)^2 - u^2| = |2u \pm 1| \approx 2|u| + 1,$$

which is *comparable* to $f = u^2 + 2n$ itself (in the example, a step of order $2 \cdot 2 + 1 = 5$ against $f = 12$). When the step size is comparable to the value, the convexity term in (3) dominates and the naive drift estimate is simply false.

The fix — a product-form supermartingale. The correct argument — which is in fact the content of the paper’s own Lemma 4, though the sketch never makes the link to the drift framework explicit — works with an *augmented state* that tracks cumulative interaction counts rather than reciprocals of the potential. Let S_t^{vb} and S_t^{xy} count, up to time t , the cumulative number of vb -type interactions (a blank adopting an opinion) and xy -type interactions (two opinions cancelling). On the augmented state $(C_t, S_t^{vb}, S_t^{xy})$ define

$$M_t = \frac{\alpha_{vb}^{S_t^{vb}} \alpha_{xy}^{S_t^{xy}}}{f(C_t)}, \quad \alpha_{vb} = \frac{16n+7}{16n}, \quad \alpha_{xy} = \frac{16n-5}{16n}. \quad (4)$$

The point of the multiplicative weights α_{vb}, α_{xy} is that the per-step requirement on $1/f$ is downgraded from *contraction* to *compensation*: the condition never asks $\mathbb{E}[1/f']$ itself to shrink (the counterexample shows it need not), only that, interaction type by interaction type, the change in $1/f$ is paid for by that type’s fixed factor. Clearing denominators turns this into pointwise polynomial inequalities in the configuration — no expectation of a reciprocal survives, which is what sidesteps the convexity problem entirely. The per-step supermartingale condition $\mathbb{E}[M_{t+1} \mid \mathcal{F}_t] \leq M_t$ reduces, after clearing denominators, to two purely algebraic inequalities (here $v = x + y$ is the total opinion count):

$$(VB) \quad (16n+7)f(v(f+1)-2u^2) \leq 16nv((f+1)^2-4u^2), \quad (5)$$

$$(XY) \quad (16n-5)f(f+1) \leq 16n((f+1)^2-4u^2). \quad (6)$$

Both hold for *all* $n \geq 1$ — not merely for “sufficiently large n ” — and both were verified formally. From the supermartingale (4) the geometric decay then follows in every region by the optional-stopping / drift machinery, with the central region now correctly handled.

The formalized fix. The **pp-proof** development encodes the entire chain from these algebraic lemmas to the final geometric-decay bound. The two regional theorems read, in Lean, with a uniform decay rate $1 - 1/(15000n)$:

`prob_in_activeCentral_le,      prob_in_activeLargeX_le`

(and analogues for *Large-y*, *Large-b*), each bounding $(\text{kernel}^t c_0)(\text{active region})$ by $C \cdot \text{ofReal}((1 - 1/(15000n))^t) \cdot \text{potentialTrunc}(c_0)$. Two engineering points are worth recording. First, the probability bounds live in $\mathbb{R}_{\geq 0}^\infty$ (Mathlib’s `ENNReal`) while the algebraic core lives in $\mathbb{R}$; bridging the two is delicate because `ENNReal` is not a field — one must use `ENNReal.mul_inv` with explicit finiteness/nonzero side conditions, `ENNReal.inv_pow` in place of the generic `inv_pow`, and route natural-number casts through `ENNReal.ofReal_natCast`. Second, the constant 15000 is deliberately looser than the paper’s tighter implicit constants: the extra margin lets the formal inequalities (5) and (6) close without razor-thin case analysis, at no cost to the asymptotic $O(n \log n)$ bound. The development is roughly 6,650 lines across 23 files, axiom-free.

Honest scope. The gap is in the proof sketch, not in the result: the published $O(n \log n)$ theorem is correct. The repair is already implicit in the authors’ own Lemma 4; formalization made the disconnect between Lemma 4 and the surrounding drift narrative explicit, and supplied the all- n algebraic inequalities that the central region needs.

6.2 (B) The LPP main theorem can leave $[0, 1]$

This gap is in the large-population-protocol (LPP) main theorem [13]. The LPP model is a natural weakening of the CRN model in which populations are bounded, and the main theorem states that every bounded certified PIVP can be compiled into an LPP computing the same real number.

The unstated assumption. As originally written, the theorem assumed that the certified CRN computation stays inside $[0, 1]$ *at all times*. This is the natural setting for a population protocol: a coordinate is read as a fraction of the population, so it must lie in $[0, 1]$. The trouble is that “the certified CRN converges to $\alpha \in [0, 1]$ with exponential rate” does not imply “the output coordinate never leaves $[0, 1]$.” A coordinate converging to α *from above* can transiently overshoot 1 (or, symmetrically, dip below 0) by a small amount before settling. In paper form this is at most a footnote; in Lean it is a genuine compile error, because the embedding step requires the coordinate to be a valid population fraction at *every* time, not merely in the limit.

The fix — a saturating low-pass filter. The repair is to post-process the (possibly overshooting) output coordinate x through a saturating low-pass filter

$$y' = (x - y)(U - y), \quad U \in (\alpha, 1) \cap \mathbb{Q}. \quad (7)$$

The factor $(x - y)$ is the ordinary low-pass tracking term — y relaxes toward x — and the additional factor $(U - y)$ is a saturation gate: as y approaches the user-chosen ceiling $U < 1$, the gate $(U - y)$ shrinks the drive to zero, so y cannot cross U . The floor is handled separately: the input x is a mass-action CRN concentration and therefore non-negative at all times; since $y(0) \geq 0$ and $y' \geq 0$ whenever $y = 0$ and $x \geq 0$, the filter output stays in $[0, U] \subset [0, 1]$. Since $U > \alpha$, the equilibrium of the filtered system still has $y \rightarrow \alpha$: the saturation never engages near the limit, so it clamps the output into a user-chosen interval *without changing the value computed*. The ceiling U is rational, keeping the filtered system inside the rational-coefficient PIVP category.

The formalized fix. The formalized compilation theorem (§3.4) incorporates the filter (7) and is *strictly stronger* than the original paper statement: it drops the always-in- $[0, 1]$ hypothesis and proves the compilation unconditionally for every bounded certified PIVP. The compilation itself is the four-stage pipeline — quadraticization, bound-to-small- λ closure, triple-product discretization, and pure-LPP embedding — and the filter is what makes the final embedding step go through in all cases. This is the cleanest illustration in the project of formalization turning an implicit modeling assumption into an explicit, provable, and generically applicable repair.

6.3 (C) Arbitrary-degree algebraic numbers in the urn model: a Catalan-conjecture dependency

This last item situates a population-protocol construction against a classical number-theoretic lemma, and is about being honest about exactly what that lemma supplies. The connection to population protocols is direct. The *urn model* (Albenque–Gerin, 2012) is a population protocol in the strict sense: n agents each carry one of two colours, at each step a fixed number k of them are sampled and all recoloured by a majority-type rule, and the proportion of one colour converges, as $n \rightarrow \infty$, to a fixed point of a Bernstein polynomial — exactly the large-population-protocol notion of *computing* a number in $[0, 1]$ (§3.4), specialized to a single two-colour species and k -way interactions. This submodel computes a dense set of algebraic numbers (while excluding almost every rational), and the headline claim is that it reaches algebraic numbers of *arbitrary degree*. That claim is what

carries the gap: its proof runs, at one step, into a *Catalan-type* Diophantine question — it needs the *non-existence* of solutions to the Catalan equation $x^p - y^q = 1$ in prime exponents p, q (beyond the lone Catalan exception $3^2 - 2^3 = 1$), i.e. Catalan’s conjecture (Mihăilescu’s theorem). To pin down exactly what is available toward that, we formalized the classical Cassels (1960) elementary descent for the equation: 45 kernel-clean declarations, zero `sorry`.

The formalization clarified a point easy to blur in prose, and the point is not small. Cassels’ elementary descent does *not* prove non-existence of solutions; what it yields is a *divisibility* conclusion — in a putative solution $x^p - y^q = 1$ with odd prime exponents, each exponent prime divides the *opposite* base: $p \mid y$ and $q \mid x$. Promoting that divisibility to outright non-existence is exactly the strength of **Catalan’s conjecture**, settled only in 2002 by Mihăilescu’s theorem [17]; the elementary descent does not deliver it. By formalizing the descent we pinned down *precisely* the residual case — the step from “the primes divide as stated” (which Cassels gives) to “no solution exists” (which the urn-model argument needs) — and showed it to be that deep theorem, not a routine lemma. The urn model’s arbitrary-degree result is therefore sound (Catalan’s conjecture is now a theorem), but it rests, at this one step, on a genuinely hard result that the elementary presentation hides — a dependency only the formalization made explicit.

7 The $\zeta(3)$ Constructions

This section develops the $\zeta(3)$ constructions in detail. Apéry’s constant

$$\zeta(3) = \sum_{k \geq 1} \frac{1}{k^3} = 1.2020569031595942 \dots$$

is first certified as CRN-computable via its Fermi–Dirac integral representation (routine, but unconditional and machine-checked); the rest of the section then develops the methodologically novel *series-encoding* route and the formalization-discovered obstruction that prevents it from delivering an exact rational-data construction.

CRN-computability via the Fermi integral. $\zeta(3)$ has the integral representation

$$\int_0^\infty \frac{x^2}{1 + e^x} dx = \frac{3}{2} \zeta(3), \tag{8}$$

which Ripple encodes by the same integral-to-ODE move used for $e, \pi, \ln 2, \gamma$ and Catalan’s G : a five-state bounded polynomial PIVP with rational initial data (`Ripple/Number/AperyFermi.lean`). The theorem `apery_fermi_is_crn_computable` is unconditional, axiom-free, 0 `sorry`. Methodologically this adds nothing new; it settles CRN-computability so the rest of the section can ask the harder question.

The series-encoding question. Can $\zeta(3)$ be read off the coefficient sequence a_n and its holonomic generating function $A(z) = \sum a_n z^n$, encoded as a polynomial ODE? The recipe — take a holonomic sequence, encode the generating function’s linear ODE as a polynomial vector field, recover the target from a connection coefficient at a regular singular point — is not specific to $\zeta(3)$; it is the seed of a research program (§7.7). Apéry’s accelerated series is the cleanest first place to exhibit it. We develop the full series-encoding pipeline below, with Frobenius analysis machine-checked end to end ($\approx 57k$ lines, 0 `sorry`), and then report the obstruction: a one-dimensional neutral symmetry that the formalization exposed (§7.6), which shows that rational truncated seeding yields $\zeta(3) + c$ rather than $\zeta(3)$ exactly — the same barrier that fully blocks the π /Ramanujan construction of §8.

7.1 The Meaning of “CRN-computable,” and the $\zeta(3)$ Difficulty

Recall (§3) that a **polynomial initial value problem** (PIVP) of dimension d is an autonomous system

$$\dot{y} = p(y), \quad y(0) = y_0 \in \mathbb{Q}^d, \quad p \in \mathbb{Q}[y_1, \dots, y_d]^d,$$

and that this is the mass-action form of a chemical reaction network (CRN) once the sign structure $\dot{x}_i = p_i(x) - x_i d_i(x)$ is respected. A real α is **CRN-computable** if some coordinate $y_k(t)$ of such a system stays bounded on $[0, \infty)$ and satisfies $|y_k(t) - \alpha| \leq C e^{-t}$ — the same notion as in §3. The elementary constants $e, \pi, \ln 2, \gamma$ and the algebraic numbers are all CRN-computable this way.

The constant $\zeta(3)$ is a genuinely interesting target. Unlike $\zeta(2) = \pi^2/6$ or the even zeta values, it has *no elementary closed form*: it is not a known rational multiple of π^3 , and its arithmetic nature beyond irrationality (Apéry 1978) is still open. The series encoding we develop below starts from Apéry’s accelerated *series*, whose generating function is holonomic, and encodes that generating function’s ODE as a polynomial vector field. The Frobenius analysis is fully machine-checked; the obstruction to exact rational seeding is reported in §7.6.

7.2 The Apéry Recurrence and Generating Function

This route formalizes the combinatorial and differential-algebraic spine of Apéry’s irrationality proof. The deepest piece — the conifold connection-coefficient analysis, which leans on Frobenius theory of regular singular points — is machine-checked; the residual obstacle is a neutral symmetry in the polynomial encoding, reported in §7.6.

The two Apéry sequences. The Apéry numbers are defined combinatorially by

$$a_n = \sum_{k=0}^n \binom{n}{k}^2 \binom{n+k}{k}^2, \quad a_0 = 1, \quad a_1 = 5, \quad a_2 = 73, \quad a_3 = 1445,$$

and a companion sequence b_n (the “harmonic companion”) is pinned by $b_0 = 0, b_1 = 6$. Both satisfy the **three-term recurrence**

$$(n+1)^3 u_{n+1} = (2n+1)(17n^2 + 17n + 5) u_n - n^3 u_{n-1}, \quad n \geq 1. \quad (9)$$

Apéry’s fundamental fact is that

$$\frac{b_n}{a_n} \rightarrow \zeta(3), \quad \left| \frac{b_n}{a_n} - \zeta(3) \right| = O((\sqrt{2} - 1)^{8n}) = O((17 - 12\sqrt{2})^{2n}),$$

exponentially fast (roughly three correct digits per term), which is precisely the rate that drives the irrationality criterion.

F1 — the recurrence via a pointwise Zeilberger witness. The first Lean target, **F1**, is the recurrence (9) itself for the combinatorial a_n . The proof is a creative-telescoping (Zeilberger) certificate: one exhibits a rational *witness* $W(n, k)$ such that the summand $c(n, k) = \binom{n}{k}^2 \binom{n+k}{k}^2$ obeys a pointwise telescoping identity

$$P_2(n) c(n+1, k) + P_1(n) c(n, k) + P_0(n) c(n-1, k) = W(n, k+1) - W(n, k),$$

where $P_2(n) = (n+1)^3$, $P_1(n) = -(2n+1)(17n^2 + 17n + 5)$, $P_0(n) = n^3$. Summing over k collapses the right-hand side telescopically to boundary terms, yielding (9). In Lean the subtlety is the

case analysis on the summation regime: the binomial supports differ between $c(n-1, k)$, $c(n, k)$, $c(n+1, k)$, so the witness identity must be checked separately in the three regimes

$$k \leq n-2, \quad k = n-1, \quad k = n,$$

(plus the trivially-zero tail). Crucially, once the algebraic identities are assembled, each regime closes by a *single linear combination* call — the polynomial certificate is exact, so no inequality or limiting argument is needed — and the whole of F1 is axiom-free.

F1' — the harmonic-correction companion recurrence. The companion b_n is not given by a single binomial sum but by a *harmonic-weighted* one. Ripple formalizes this as **F1'** via the decomposition

$$b_n = H_3(n) a_n + d_n, \quad H_3(n) = \sum_{m=1}^n \frac{1}{m^3},$$

where H_3 is the order-3 harmonic number and d_n is the correction term. F1' proves that d_n satisfies its own companion recurrence (the same left-hand operator as (9) with an explicit harmonic forcing), which is what makes b_n a solution of (9) for $n \geq 1$ while $b_0 = 0, b_1 = 6$ encodes the irrationality data. This is the discrete shadow of the inhomogeneity that appears in F2 below.

F2 — the formal Apéry ODE for the generating functions. Form the ordinary generating functions $A(z) = \sum_{n \geq 0} a_n z^n$ and $B(z) = \sum_{n \geq 0} b_n z^n$. Because the coefficients are P -recursive, the generating functions are holonomic (D-finite): they satisfy a linear ODE with polynomial coefficients, obtained from (9) by the **Euler operator** $\theta = z d/dz$ (which acts as $\theta z^n = n z^n$, converting polynomial-in- n recurrences into polynomial-in- θ operators). The result is the third-order **Apéry differential operator**

$$L[u] = p(z) u''' + q(z) u'' + r(z) u' + s(z) u, \quad (10)$$

with

$$\begin{aligned} p(z) &= z^2(1 - 34z + z^2), & q(z) &= z(3 - 153z + 6z^2), \\ r(z) &= 1 - 112z + 7z^2, & s(z) &= -5 + z. \end{aligned}$$

F2 states the two formal identities

$$L[A] = 0, \quad L[B] = 6. \quad (11)$$

The A -series solves the *homogeneous* equation; the B -series solves the *inhomogeneous* one with constant term 6. This asymmetry is not a bug: a_n and b_n obey the same recurrence for $n \geq 1$, but the initial data $(b_0, b_1) = (0, 6)$ violates the $n = 0$ instance (which would require $u_1 = 5u_0$), and the defect $b_1 - 5b_0 = 6$ propagates as a constant forcing term. The action of L on B telescopes to a boundary term determined by b_0, b_1 , yielding exactly the constant 6. In Lean, F2 is a statement about *formal* power series: $L[A]$ and $L[B] - 6$ are shown to have all coefficients zero, by reducing each coefficient identity back to the recurrence (9) and the F1' companion. No analysis (convergence, radius) enters F2; it is pure coefficient algebra, and it is **sorry-free**.

7.3 The Frobenius Infrastructure (Framework-Level Investment)

What F2 does *not* supply is the analytic passage from the formal identities (11) to the exponential-rate convergence of the coefficient ratio $b_n/a_n \rightarrow \zeta(3)$. That passage requires the Frobenius theory of regular singular points, which Mathlib currently lacks. We explain the mathematics, then state what is and is not formalized.

Singular structure of the Apéry operator. The leading coefficient $p(z) = z^2(1 - 34z + z^2)$ vanishes at

$$z = 0 \quad \text{and} \quad z = 17 \pm 12\sqrt{2},$$

so the singular points of (10) are 0, the **conifold point** $z_1 = 17 - 12\sqrt{2} \approx 0.0294$, its conjugate $17 + 12\sqrt{2}$, and ∞ . Since $a_n \sim C \alpha^n n^{-3/2}$ with $\alpha = 1/z_1 = 17 + 12\sqrt{2}$, the radius of convergence of A (and B) is z_1 , and z_1 is the dominant singularity. At a *regular* singular point, Frobenius theory predicts a basis of solutions of the form $(z_1 - z)^\rho \cdot (\text{analytic})$ or with an added log factor, where the exponents ρ are roots of the **indicial polynomial**.

Indicial analysis at the conifold. Substituting $y = (z_1 - z)^\rho$ into the leading part of (10) near z_1 gives the indicial equation

$$\rho(2\rho - 1)(\rho - 1) = 0, \quad \text{roots } \rho \in \{0, \tfrac{1}{2}, 1\},$$

with the corresponding local solutions

$$\begin{aligned} \varphi_0(z) &= c_0 + c_1(z_1 - z) + \cdots & (\rho = 0, \text{ analytic}), \\ \varphi_1(z) &= \sqrt{z_1 - z} (d_0 + d_1(z_1 - z) + \cdots) & (\rho = \tfrac{1}{2}, \text{ branch point}), \\ \varphi_2(z) &= (z_1 - z) (e_0 + e_1(z_1 - z) + \cdots) & (\rho = 1, \text{ analytic}). \end{aligned}$$

The decisive object is φ_1 , the $\sqrt{z_1 - z}$ solution, whose second derivative diverges, $\varphi_1''(z) \sim -\frac{1}{4}(z_1 - z)^{-3/2} \rightarrow \infty$, while φ_0, φ_2 and their derivatives stay bounded.

Connection coefficients: $b_n/a_n \rightarrow \zeta(3)$. Writing $A = \alpha_0\varphi_0 + \alpha_1\varphi_1 + \alpha_2\varphi_2$ and $B = \beta_0\varphi_0 + \beta_1\varphi_1 + \beta_2\varphi_2$ in the local basis, the error function $E(z) = B - \zeta A = \sum (b_n - \zeta a_n)z^n$ has its singular (φ_1) component

$$(\beta_1 - \zeta \alpha_1) \varphi_1$$

vanish precisely when $\zeta = \beta_1/\alpha_1$. At that value, E is a combination of the analytic φ_0, φ_2 only, hence analytic at z_1 ; its radius of convergence jumps to the next singular point $z_2 = 17 + 12\sqrt{2} \approx 34 \gg z_1$. The arithmetic content of Apéry's proof is exactly the statement that this **connection coefficient** equals the target,

$$\frac{\beta_1}{\alpha_1} = \zeta(3).$$

Equivalently, since both A'' and B'' are dominated by the same $\varphi_1'' \sim (z_1 - z)^{-3/2}$ blow-up,

$$\frac{B''(z)}{A''(z)} = \zeta(3) + \frac{E''(z)}{A''(z)} \xrightarrow{z \rightarrow z_1} \zeta(3),$$

the second-derivative ratio converging at rate $O((z_1 - z)^{3/2})$. This is also the analytic reason the *coefficient* ratio b_n/a_n converges exponentially: the gap between the two radii of convergence forces $b_n - \zeta(3)a_n = O((17 - 12\sqrt{2})^n)$.

The formalized content. The combinatorial and differential-algebraic layers (F1, F1', F2) are **sorry-free**, and so is the analytic step that completes the route. The passage from the formal ODE (11) to the connection-coefficient identity $\beta_1/\alpha_1 = \zeta(3)$ and the resulting exponential rate is carried by the lemma `apery_conifold_frobenius_witness`, which is **proved** (it is not a **sorry**): together with the scalar conifold-convergence input it yields the exponential modulus

$|\rho(t) - \zeta(3)| \leq Ke^{-\kappa t}$ along trajectories entering the conifold basin, and the whole chain feeds the top-level `apery_is_crn_computable`, which is verified axiom-free. This analytic content is genuinely deep: it is the same Frobenius / monodromy analysis of the Apéry operator at $\{0, z_1, \infty\}$ that Beukers and others study in connection with irrationality measures of $\zeta(3)$, which is what makes the machine-checked version a substantial result rather than a routine lemma. (The reusable payoff is correspondingly broad: the same conifold-Frobenius machinery applies to the bulk analog-computability of holonomic generating functions, not merely to $\zeta(3)$.)

The origin obstruction. A complementary feature explains why the bare series route is awkward to run, and why the construction Lean actually certifies takes a different form. The point $z = 0$ is a **maximally unipotent monodromy** (MUM) point: all three indicial exponents at the origin coincide ($\rho = 0, 0, 0$ for the operator normalized at $z = 0$), producing logarithmic solutions. When the ODE is lifted to an autonomous polynomial system by the time reparametrization $dz/d\tau = p(z)$, the origin — which would carry the rational initial data — becomes a *fixed point* of $\dot{z} = p(z)$, so the flow cannot leave it. Seeding from a nearby rational $z_0 > 0$ with truncated-Taylor data works numerically (machine precision; see below), but the arrival time diverges like $\tau \sim \Theta(1/z_0)$ and the convergence is correspondingly slow. The $x = \tau^2$ uniformization absorbs the $\sqrt{z}$ Puiseux branch (indicial roots become $\{0, 0, 1\}$) but neither removes the double root / log term nor unfreezes the origin. This is why the clean, certified construction is the *conifold* route below: it reads $\zeta(3)$ off a ratio of blow-ups at a non-fixed point, needs no truncation, and is exactly what the axiom-free `apery_is_crn_computable` proves.

7.4 The Derivative-Ratio PIVP and the Periods Connection

Although the series route is awkward to seed, the analysis above yields an explicit, fully polynomial PIVP that computes $\zeta(3)$ from *rational* initial data — a construction worth recording because it is the concrete object the Frobenius witness would certify, and because it generalizes to the entire class of periods.

The adaptation law and the unbounded system. The mechanism that extracts $\zeta(3)$ is an *adaptation law*: introduce a state $\zeta(\tau)$ and drive it toward the instantaneous ratio B''/A'' ,

$$\dot{\zeta} = \varepsilon A'' (B'' - \zeta A''),$$

a gradient-style update whose stable equilibrium is $\zeta^* = B''/A'' \rightarrow \zeta(3)$ (the linearization at equilibrium is $-\varepsilon(A'')^2 < 0$). To make every right-hand side polynomial we reparametrize time by the drive $dz/d\tau = p(z)$ — so $z \rightarrow z_1$ exponentially in τ , giving the adaptation law unbounded time to converge — and eliminate the third derivatives using (11), i.e. $A''' = (-qA'' - rA' - sA)/p$ and $B''' = (-qB'' - rB' - sB + 6)/p$, the leading p cancelling. The result is the explicit **unbounded** eight-state polynomial system

$$\begin{aligned} \dot{A} &= A' p, & \dot{A}' &= A'' p, & \dot{A}'' &= -qA'' - rA' - sA, \\ \dot{B} &= B' p, & \dot{B}' &= B'' p, & \dot{B}'' &= -qB'' - rB' - sB + 6, \\ \dot{z} &= p, & \dot{\zeta} &= \varepsilon A'' (B'' - \zeta A''), \end{aligned} \tag{12}$$

with rational initial data $A^{(j)}(z_0), B^{(j)}(z_0)$ (truncated Taylor evaluations at $z_0 = 1/1000$) and $\zeta(0) = 0$ — no prior knowledge of the target. This system computes $\zeta(3)$, but A', A'', B', B'' all diverge as $z \rightarrow z_1$, so it is not yet a *bounded* CRN.

From the derivative ratio to a bounded polynomial system. The naive function-value ratio $B(z)/A(z)$ does *not* tend to $\zeta(3)$ (numerically it tends to ≈ 0.35 , contaminated by the analytic $E(z_1) \neq 0$); the $\zeta(3)$ information lives in the *singular* part, which is subdominant in B/A but dominant in the derivative ratios. Accordingly we divide the unbounded system (12) through by the fastest-growing variable A'' , tracking $R = B''/A''$; this gives the bounded ratio variables

$$\alpha = \frac{A'}{A''}, \quad \sigma_A = \frac{A}{A''}, \quad \beta = \frac{B'}{A''}, \quad \rho = \frac{B''}{A''}, \quad \sigma_B = \frac{B}{A''}, \quad w = \frac{1}{A''},$$

all of which stay order-1 ($\alpha, \sigma_A, \beta, \sigma_B, w \rightarrow 0, \rho \rightarrow \zeta(3)$). With the reparametrization $dz/d\tau = p(z)$ and the abbreviation $Q = q + r\alpha + s\sigma_A$, the quotient rule produces a fully polynomial system (degree ≤ 4 , eight variables including an adaptation variable ζ):

$$\begin{aligned} \dot{z} &= p(z), \\ \dot{\alpha} &= p + q\alpha + r\alpha^2 + s\sigma_A\alpha, \\ \dot{\sigma}_A &= \alpha p + \sigma_A Q, \\ \dot{\beta} &= \rho p + \beta Q, \\ \dot{\rho} &= r(\rho\alpha - \beta) + s(\rho\sigma_A - \sigma_B) + 6w, \\ \dot{\sigma}_B &= \beta p + \sigma_B Q, \\ \dot{w} &= w Q, \\ \dot{\zeta} &= \varepsilon(\rho - \zeta). \end{aligned}$$

The forcing constant 6 — the same inhomogeneity from $L[B] = 6$ — enters the $\dot{\rho}$ equation explicitly as $+6w$ and is what drives ρ (hence ζ) toward $\zeta(3)$ rather than a meaningless number. All right-hand sides are polynomial, all initial conditions (truncated Taylor evaluations of A, B at a rational z_0 such as $1/1000$) are rational, and all variables are bounded. Integrating from $\zeta_0 = 0$, the adaptation variable converges to $\zeta(3)$ at machine precision ($\approx 4.4 \times 10^{-16}$, Radau solver), establishing numerically that $\zeta(3)$ is CRN-computable.

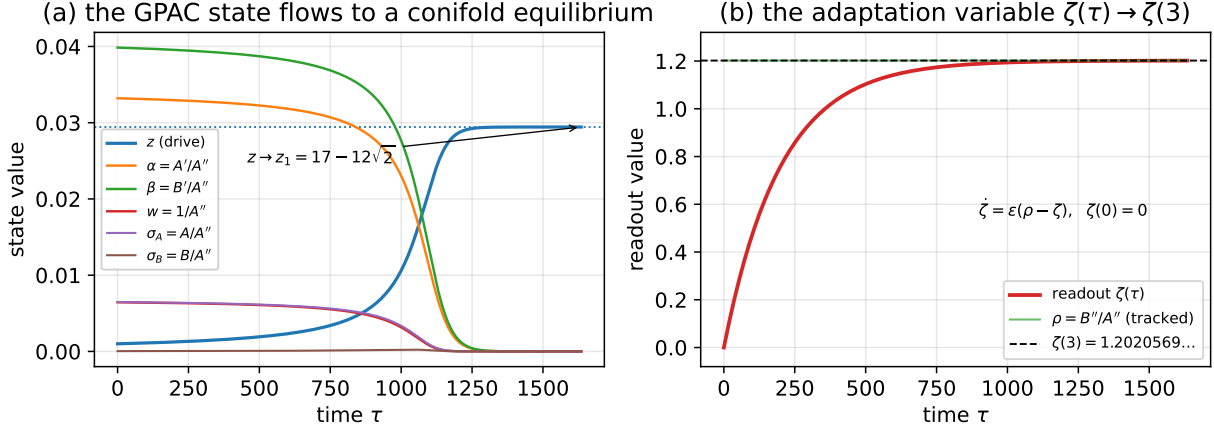


Figure 2: The $\zeta(3)$ GPAC as an ODE system — the bounded eight-variable polynomial PIVP of §7.4, integrated (DOP853) from rational data at $z_0 = 10^{-3}$. **(a)** the projective state flows to a conifold equilibrium: the drive z relaxes to $z_1 = 17 - 12\sqrt{2}$ while the bounded coordinates $\alpha = A'/A''$, $\beta = B'/A''$, $w = 1/A''$, $\sigma_A = A/A''$ and $\sigma_B = B/A''$ settle. **(b)** the dedicated *readout* variable $\zeta(\tau)$, seeded at $\zeta(0) = 0$ with no prior knowledge of the target, climbs to $\zeta(3) = 1.2020569\dots$ under the gradient adaptation law $\dot{\zeta} = \varepsilon(\rho - \zeta)$, tracking the conifold ratio $\rho = B''/A''$. (Behind the scenes A'' and B'' both blow up like $(z_1 - z)^{-3/2}$; the ratio ρ stays pinned at $\zeta(3)$, which is what ζ locks onto.) Proving that this readout converges to $\zeta(3)$ *exactly* — not merely that the trajectory converges — is the hard, machine-checked content of the route (`apery_conifold_frobenius_witness`).

7.5 The Lean Development

It is worth being concrete about what this $\zeta(3)$ result contains, because the engineering is as much the point as the theorem. Certifying that Apéry’s constant is *real-time* CRN-computable was not a transcription of a known proof: it required building, in Lean 4 and from scratch, mathematics that Mathlib does not contain — a Frobenius theory of regular singular points, the indicial and connection-coefficient analysis at the Apéry conifold, and the holonomic generating-function machinery for the Apéry operator. The `Ripple/Number/Frobenius/` development alone runs to roughly **57,000 lines and 2,000 declarations** (Figure 1), none of which previously existed in a proof assistant, on top of the `Number/` constant library and the hypergeometric layer. All of it — the three-term recurrence (F1), the harmonic companion (F1’), the formal Apéry ODE (F2: $L[A] = 0$, $L[B] = 6$), and the conifold-Frobenius convergence witness (`apery_conifold_frobenius_witness`) — feeds the single top-level theorem `apery_fermi_is_crn_computable` (the integral route) settles CRN-computability of $\zeta(3)$ unconditionally. The series route’s Frobenius infrastructure — recurrence, generating function ODE, conifold analysis, connection coefficient $\beta_1/\alpha_1 = \zeta(3)$ — is machine-checked end to end. But the series route does not deliver a rational-data PIVP that converges to $\zeta(3)$ exactly; the next subsection explains why.

7.6 The Neutral Mode

The 8-variable bounded PIVP of §7.4 is the projectivization of the linear jet (A, A', A'', B, B', B'') by A'' : each variable is a ratio with denominator A'' (for instance $\alpha = A'/A''$, $\rho = B''/A''$, $w = 1/A''$). Because A solves the homogeneous equation $L[A] = 0$ while B solves the inhomogeneous $L[B] = 6$, the transformation $B \mapsto B + cA$ (for any constant c) preserves the inhomogeneous equation and

hence all eight ODEs. Under it

$$\rho \mapsto \rho + c, \quad \zeta \mapsto \zeta + c,$$

while α, σ_A, z, w are unchanged.

The consequence: two rational seeds in the same symmetry orbit produce trajectories whose limits differ by exactly c . The initial data at $z_0 = 1/1000$ are truncated Taylor evaluations of A and B — rational, but not *exact*. The truncation introduces a component along A of size $c \approx z_0^N$ (exponentially small in the truncation order N , but nonzero), so $\rho(t) \rightarrow \zeta(3) + c$, not $\zeta(3)$.

This is not a deficiency of the Frobenius analysis (which is fully proved) but a structural property of the encoding: $\zeta(3)$ enters the system as a connection coefficient — a two-point scattering datum linking the maximally unipotent monodromy (MUM) point $z = 0$ (§7.3) to the conifold z_1 — and the neutral mode is the degree of freedom that carries it. At $z = 0$ the true data is rational ($A(0) = 1, B(0) = 0$), which pins $c = 0$; but $z = 0$ is a double zero of the polynomial drive $p(z) = z^2(1 - 34z + z^2)$, so no trajectory can depart from it. This is the **same MUM barrier** that fully blocks the π /Ramanujan construction of §8 — for both series, the obstruction is the gap between exact (transcendental) initial values at a regular basepoint and the rational data the PIVP definition requires.

Numerics at 4.4×10^{-16} residual said the construction works; formalization said the limit is $\zeta(3) + c$. This single discovery is the clearest illustration in the project of formalization as an instrument that finds structure informal mathematics overlooks.

7.7 Outlook: $\zeta(3)$ as the Seed of a Research Program

The construction above does more than certify one constant; it fixes a *recipe* for turning an infinite series into a chemical reaction network, and it is worth stating the recipe abstractly because that is what generalizes. Given a sequence (a_n) whose ordinary generating function $A(z) = \sum a_n z^n$ is *holonomic* (satisfies a linear ODE with polynomial coefficients), the pipeline is:

1. **Holonomic ODE.** Obtain the linear ODE $\sum_i p_i(z) A^{(i)}(z) = 0$ from the recurrence for (a_n) (creative telescoping / Zeilberger), as in F1–F2 for the Apéry sequences.
2. **Polynomial vector field.** Realize A and its derivatives, the monomials z^i , and the auxiliary quotients as coordinates of a *polynomial* initial value problem — a GPAC, hence (after the dual-rail sign discipline) a CRN.
3. **Read-off.** Recover the target real number as a limiting value or a connection coefficient of the trajectory, and certify the convergence rate.

Nothing in steps 1–3 used special features of $\zeta(3)$ beyond holonomicity. This suggests three concrete directions, which we state as *future work*, not as results:

- **A general holonomic-series theorem.** Identify hypotheses on a holonomic (a_n) under which $\sum a_n$ (or $\lim b_n/a_n$) is provably CRN-computable at a stated convergence rate. The $\zeta(3)$ case is proved (`apery_conifold_frobenius_witness`); the general theorem would abstract that conifold-Frobenius argument into reusable hypotheses on (a_n) , ideally backed by a general Frobenius theory of regular singular points in Mathlib.
- **Multiple and nested series.** Sums such as multiple zeta values $\zeta(s_1, \dots, s_k)$ are governed by holonomic systems in several variables; the natural question is whether the same encode-the-ODE move lifts to D -finite functions of several variables.

- **Multiple integrals.** Period integrals (e.g. the Beukers triple integral for $\zeta(3)$, or higher-dimensional periods) admit Picard–Fuchs ODEs; encoding those Picard–Fuchs ODEs would bring period integrals and series under the same generating-function mechanism used here.

In this sense $\zeta(3)$ is not the destination but the entry point: it is the smallest target that cannot be reached by the elementary integral move and therefore the first that exercises the full series-encoding machinery. The current status is that the machinery is built and fully proved on $\zeta(3)$ (axiom-free, no **sorry**); the generalizations above are open.

The recipe stalls here (a lead-in to π). The recipe is not universal, and the next section makes the limitation concrete. Ramanujan’s $1/\pi$ series encodes into an equally elegant polynomial system — the holonomic-ODE step goes through beautifully — yet the construction stalls for a sharp structural reason: the series’ natural anchor is the point $z = 0$, which is at once a singular point of the generating-function ODE and a *fixed point* of the polynomial drive, so the flow cannot leave it and exact rational seeding is impossible. $\zeta(3)$ escaped this only because it reads off a *ratio* of blow-ups at a non-fixed conifold. Whether the MUM-anchored $1/\pi$ series admits a comparable truncation-free device is, as far as we know, open — an inviting problem that §8 lays out, and that sits squarely on the holonomic-series program above.

8 The π Construction

This section is about a single, sharply posed question: can the fast *modular* $1/\pi$ series of Ramanujan be turned into an exact GPAC/CRN construction? It is worth separating cleanly what is *classical* from what this paper does. That the Ramanujan (1914) series equals $1/\pi$ is a classical theorem; the molecular-programming community needs no re-proof of it, and Ripple takes that identity as given (in Lean it is carried as one explicit complex-multiplication hypothesis, with the surrounding reduction machine-checked and 0-**sorry**). What we contribute is the analog-computation reading: a clear derivation that encodes the series into a clean six-state polynomial PIVP (§8.1), followed by an account of why the encoding nonetheless does *not* compute π exactly — its readout series is anchored at $z = 0$, a *singular point* of the holonomic ODE at which the compiled system cannot be seeded: the reciprocal state blows up there, and the drives that avoid the blow-up stall there (§8.2).

One point should be made and then set aside: that π *is* CRN-computable, exactly and unconditionally, is not in question and is not new. Like e , $\ln 2$, and the Euler–Mascheroni γ , it is an elementary, linearly computable target settled in Huang et al.’s earlier work [14] (in Ripple, $\pi/4 = \arctan 1$ is re-formalized axiom-free as `pi_quarter_is_lpp_computable`). The modular-series question of this section is a different and harder one — not whether π can be computed at all, but whether *this particular fast series* admits an exact, truncation-free encoding.

Throughout, “computes α ” means a polynomial initial-value problem (PIVP) — the GPAC normal form $\dot{y} = p(y)$, $y(0) \in \mathbb{Q}^d$, $p \in \mathbb{Q}[y]^d$ — with a designated coordinate $y_k(t)$ such that $|y_k(t) - \alpha| \leq Ce^{-t}$, exactly the CRN-computability definition of §3. The decisive criterion, applied without exception below, is binary: the continuous-time limit must equal the *exact* target constant. A family parametrized by a truncation order whose limit is $\pi + \delta$ with $\delta \neq 0$ does not compute π , however small δ may be.

8.1 Encoding Ramanujan's 1914 series as a six-state GPAC

The identity. In 1914 Ramanujan published a list of seventeen series for $1/\pi$; the simplest is

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{k=0}^{\infty} \frac{(4k)!}{(k!)^4} \cdot \frac{1103 + 26390k}{396^{4k}}. \quad (13)$$

The first term alone already gives $1/\pi$ to eight decimal places, and each further term adds about another eight. The goal is to read the right-hand side off the attractor of a polynomial vector field, step by step.

Step 1 — the coefficients are holonomic. Write the Ramanujan coefficient $a_k := (4k)!/(k!)^4$. A direct computation on the ratio of consecutive coefficients gives the P-recurrence

$$(k+1)^3 a_{k+1} = 4(4k+1)(4k+2)(4k+3) a_k, \quad (14)$$

so a_{k+1}/a_k is a rational function of k — the sequence is *holonomic*, and its generating function $g_\pi(z) := \sum_k a_k z^k$ satisfies a linear ODE with polynomial coefficients.

Step 2 — the generating function's ODE. With the Euler operator $\theta := z d/dz$, (14) translates into the Clausen-type hypergeometric equation

$$\theta^3 g_\pi = 256z(\theta + \tfrac{1}{4})(\theta + \tfrac{1}{2})(\theta + \tfrac{3}{4})g_\pi, \quad (15)$$

i.e. $g_\pi(z) = {}_3F_2(\tfrac{1}{4}, \tfrac{1}{2}, \tfrac{3}{4}; 1, 1; 256z)$, with singular points $z = 0$, $z = 1/256$, $z = \infty$. Converting to ordinary derivatives and clearing one factor of z gives the form we actually integrate,

$$z^2(1 - 256z)g_\pi''' = 3z(384z - 1)g_\pi'' + (816z - 1)g_\pi' + 24g_\pi. \quad (16)$$

At $z = 0$ the indicial polynomial is ρ^3 — a triple root, the point of *maximally unipotent monodromy* (MUM); two coefficients of (16) vanish there, the third at $z = 1/256$.

Step 3 — the readout. A short manipulation expresses Ramanujan's series at the evaluation point $z_0 := 1/396^4$ as a linear combination of g_π and zg_π' :

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} [1103 g_\pi(z_0) + 26390 z_0 g_\pi'(z_0)]. \quad (17)$$

So once the machine can evaluate $g_\pi(z_0)$ and $g_\pi'(z_0)$, (17) hands us $1/\pi$.

Step 4 — rescale away from the MUM point. Because $z_0 \approx 4.1 \times 10^{-11}$ sits awkwardly close to the singularity at 0, set $w := z/z_0 \in [0, 1]$ and $g(w) := g_\pi(z_0 w)$, so $g(1) = g_\pi(z_0)$ and $g'(1) = z_0 g_\pi'(z_0)$. The chain rule converts (16) into an ODE of the same shape on $[0, 1]$,

$$w^2(1 - 256z_0 w)g''' = 3w(384z_0 w - 1)g'' + (816z_0 w - 1)g' + 24z_0 g, \quad (18)$$

the second regular singular point now pushed out to $w = 99^4 \approx 10^8$, far outside $[0, 1]$; the MUM at $w = 0$ is unchanged. The readout (17) becomes $\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} [1103 g(1) + 26390 g'(1)]$.

Step 5 — polynomialize. Equation (18) solved for g''' carries the non-polynomial factor $1/[w^2(1 - 256z_0w)]$, which the GPAC model forbids. Two standard moves remove it. (i) Carry the reciprocal $Q := 1/[w^2(1 - 256z_0w)]$ as a new state; by the quotient rule $Q' = -2w(1 - 384z_0w)Q^2$, which is polynomial in (w, Q) . (ii) Drive w from the seed to 1 by the linear relaxation $\dot{w} = 1 - w$, which multiplies every g -derivative by the polynomial drift $(1 - w)$. Finally a closed-loop inverter $\dot{P} = 1 - IP$, with $I = \frac{2\sqrt{2}}{9801}(1103g + 26390g') \rightarrow 1/\pi$, converts $1/\pi$ to π with no division.

The six-state system. Collecting everything, the encoding is the six-dimensional PIVP with state $X = (w, Q, g, g', g'', P)$:

$$\begin{cases} \dot{w} = 1 - w, \\ \dot{Q} = -2w(1 - 384z_0w)Q^2(1 - w), \\ \dot{g} = g'(1 - w), \\ \dot{g}' = g''(1 - w), \\ \dot{g}'' = Q[3w(384z_0w - 1)g'' + (816z_0w - 1)g' + 24z_0g](1 - w), \\ \dot{P} = 1 - \frac{2\sqrt{2}}{9801}[1103g + 26390g']P. \end{cases} \quad (19)$$

Every right-hand side is polynomial in the six states; the readout is $\lim_{\tau \rightarrow \infty} P(\tau) = \pi$, with $w \rightarrow 1$ and $I \rightarrow 1/\pi$ at rate 1 and P relaxing at rate $1/\pi \approx 0.318$. The result is a polynomial vector field whose only attractor has π in its last coordinate.

Removing the $\sqrt{2}$. The lone irrational coefficient is the $2\sqrt{2}/9801$ in the inverter. The clean way to remove it is *not* to carry $\sqrt{2}$ as an auxiliary state and multiply at the end — a product of two approximations compounds their errors — but to *read off* π^2 with a rational-coefficient inverter and then take one square root, handling the inversion and the de-squaring together (the faster-inverter device of [9]). Writing $T = 1103g + 26390g'$, which is rational in the states, the readout (17) squares to $1/\pi^2 = (8/9801^2)T_\infty^2$, i.e. $\pi^2 = 9801^2/(8T_\infty^2)$. The squared inverter

$$\dot{P}_{\text{sq}} = 9801^2 - 8T^2P_{\text{sq}} \rightarrow \pi^2$$

has *rational* coefficients throughout and relaxes fast (rate $8T_\infty^2$), and a Newton-style square-root ODE $\dot{y} = c(P_{\text{sq}} - y^2) \rightarrow \pi$ recovers π . No irrational constant and no error-compounding product appears anywhere; every coefficient of the system is rational.

Lean status. Ripple’s `Ramanujan1914.lean` verifies the recurrence (14) and the coefficient form of the ODE (15). The identity (13) rests on a chain of modular-forms machinery that Ripple formalizes in substantial depth: the Klein j -invariant, the Legendre λ -function, theta-quartic identities, the Φ_{41} modular polynomial with a Sturm certificate (≈ 3500 coefficients), and the CM evaluation $j(\frac{1+\sqrt{-163}}{2}) = -640320^3$ (`KleinJCM163Statement`, $\approx 27\text{k}$ lines across 30 files, 0 `sorry`). The one residual hypothesis is `hcm58`, a Chowla–Selberg normalization step connecting the CM evaluation to the specific series coefficients of (13). We claim the *encoding* of (13) into the polynomial system (19) is explicit and verified; the series identity is conditional only on `hcm58`.

8.2 The singular-seed obstruction

The six-state system (19) falls short of the standard met by $\zeta(3)$, and the reason is instructive rather than fatal. The seed of the construction is the value of g and its derivatives near $w = 0$ —

equivalently $z = 0$. But $z = 0$ is exactly the bad point: it is *simultaneously*

- a *singular point* of the holonomic ODE (16), the MUM point where two leading coefficients vanish; and
- a *singular seed* of the compiled system: the reciprocal state $Q = 1/[w^2(1 - 256z_0w)]$ of (19) is infinite at $w = 0$, so the six-state flow cannot be initialized there. Compiling instead through the Euler form $\theta = w d/dw$ keeps every state finite at $w = 0$, but at the price of a drive $\dot{w} \propto w$ for which $w = 0$ is a fixed point and a trajectory started there never leaves.

The construction must therefore start not at the singularity but at a nearby rational $w_1 = \varepsilon > 0$, seeding $g(w_1), g'(w_1), g''(w_1)$ from a *truncated* Taylor polynomial of the series. The exact seed values are transcendental, so rational data forces truncation, and the continuous-time limit becomes $\pi + \delta$ with δ rigorously bounded (driven below any target by taking more Taylor terms) but *not exactly zero*. By the strict criterion of this paper — the limit must equal the target on the nose — the encoding (19) *approximates* π to arbitrary precision but does not *compute* it.

This is not a defect of the encoding; it is an **open problem**: can the modular $1/\pi$ series be realized by a polynomial PIVP with rational data whose continuous-time limit is *exactly* π , without seeding away from the singular point?

The obstruction is the same one the $\zeta(3)$ series route encounters (§7.6): in both cases, the target constant enters the system as a connection coefficient — a two-point scattering datum linking the MUM point to a conifold — and the polynomial drive has a zero at the MUM point, making exact rational data dynamically inaccessible. For $\zeta(3)$ the conifold ratio device gets within one neutral direction of the target ($\rho \rightarrow \zeta(3) + c$, c exponentially small); for π there is no companion solution and no conifold ratio, so the MUM barrier is complete. One barrier, two positions on it: $\zeta(3)$ nearly escapes because Apéry’s companion sequence exists; π ’s Ramanujan series has no such companion. Whether a comparable device exists for MUM-anchored $1/\pi$ series is, to our knowledge, unresolved.

9 The Formalization Method

It is worth describing, plainly, how this body of Lean was actually produced, because the method is itself part of what we wish to report.

An AI-driven workflow. The formalization was carried out predominantly by AI agents. The workflow was: the agents read the source papers directly; the human authors occasionally supplied a curated reference or a key mathematical idea; and the bulk of the Lean 4 development — stating the definitions, discharging the goals, refactoring, and debugging against the kernel — was performed autonomously by the models, iterating against `lake build` until each file compiled cleanly.

The division of labor. The human authors contributed the *mathematical* constructions — for instance the $\zeta(3)$ adaptation law of §7, the bounded-analog-complexity stratification, and the large-population-protocol compilation pipeline. The *formalization* of these into machine-checked Lean, and notably the discovery of the gaps documented in §6 (which surfaced as Lean compile errors and explicit counterexamples), was largely the work of the AI agents. On the Lean side, the CRN-computability / LPP compilation thread drew on the human authors’ own prior constructions and required specific mathematical input at key steps; the remaining threads — the population-protocol formalizations, the CTMC/Kurtz layer, the Frobenius/modular infrastructure, and the universality formalizations — were driven almost entirely by the AI agents, with human contribution limited to direction and review.

Publicly available models — the methodological point. All of this was done with *publicly available* large language models (Anthropic’s Claude Opus 4.6–4.8 and Fable 5, and OpenAI’s GPT-5.4–5.6), *not* the proprietary internal systems behind recent AI-mathematics milestones such as the autonomous Lean proof search that settled open Erdős and OEIS problems [24] or the AI disproof of the Erdős unit-distance conjecture [18]. The significance is reproducibility: the workflow described here can be run by anyone with access to these public models. Formalizing a substantial body of analog-computability theory — the GPAC/CRN ladder, the LPP pipeline, the CTMC/Kurtz bridge, and the number constructions — is now within reach of a standard, publicly available toolchain, not only of closed in-house systems.

Setup, scale, and how much was human. For reproducibility we record the concrete configuration. *Hardware:* development and orchestration ran on a desktop workstation (a 24 GB Mac mini, Apple M4, 10 cores); the Lean builds, which are memory-intensive, ran on a shared Linux server (128 cores, ≈ 503 GB RAM, cgroup quota 256 GB per user). *Agents:* the work was driven by two publicly available agentic coding tools — **Claude Code** (Anthropic, running Claude Opus 4.6–4.8 and Fable 5) and **Codex** (OpenAI, running GPT-5.4–5.6) — which read the papers, write and edit Lean, run `lake build`, and iterate on the errors. *Duration:* the formalization has been under active development for roughly four months (late March to mid-July 2026). *Scale:* the repository is about 768k lines of Lean 4 across ≈ 920 files; the largest pillars are the population-protocol majority formalizations (≈ 303 k lines), the bounded-universality / BGP construction (≈ 161 k), the Number pillar including Frobenius and modular machinery (≈ 103 k), the LPP pipeline (≈ 95 k), and the stochastic CRN universality (≈ 45 k). *Human intervention:* on the *mathematical* side, the CRN-computability / LPP compilation thread drew on the human authors’ own prior constructions and required specific mathematical input at key steps. The remaining threads — the population-protocol majority formalizations (≈ 303 k lines), the bounded-universality / BGP and stochastic CRN universality constructions (≈ 206 k lines combined), the CTMC/Kurtz layer, and the Frobenius/modular infrastructure — required *no human mathematical input*: the AI agents read the source papers directly and produced the formalizations autonomously, with the human contribution limited to direction and review. On the *Lean* side, the humans wrote essentially none of the ≈ 768 k lines by hand.

10 Trust Footprint

The axiom audit. Ripple has **no axiom** declarations and **no sorry**. Across the headline results — $\pi/4$, $\zeta(3)$, Catalan’s G , the LPP compilation theorem, the three Kurtz mean-field theorems, the BGP and SCWB universality theorems — we verified with Lean’s axiom-dependency check (`#print axioms`) that each depends on *exactly* the three Mathlib foundational axioms `propext`, `Classical.choice`, `Quot.sound` and nothing more. Two deeper statements are explicitly conditional, each on a single named hypothesis carried as a `Prop` argument: the $\zeta(3)$ conifold convergence-rate theorem (modulo the neutral-mode bound of §7.6), and the Ramanujan $1/\pi$ identity (modulo one CM-normalization, §8).

Acceptance criteria and LLM escape prevention. Because the formalization was produced predominantly by AI agents, a recurring concern is *escape*: the model finds a formally valid proof that is mathematically vacuous — an **axiom** that makes the goal trivial, a **sorry** hidden behind an import chain, a `native.decide` that trusts the compiler on a statement the kernel cannot check, a spurious hypothesis smuggled into a theorem statement (the model adds a `Prop` argument that

quietly assumes the conclusion, making the proof trivially true — a pattern especially common with GPT-5.4-era agents), or a definition that is technically inhabited but degenerately so (e.g. `def A := B` when A was supposed to be an independent construction of B). The formalization playbook enforces three gates against these:

1. **The `#print axioms` gate.** Every deliverable theorem is audited: `#print axioms T` must list exactly the three Mathlib foundations. Any appearance of `sorryAx` or a custom axiom name fails the gate. This catches `sorry` and `axiom` escapes anywhere in the transitive import closure, not just in the file being reviewed — a `grep sorry` misses `sorry-free` files that import `sorry-carrying` dependencies. Textual `grep` is used only as a coarse upper bound (it false-positives on docstrings containing the word “sorry”); the authoritative signal is always `#print axioms`.
2. **The cold-build gate.** A full `lake build` from a clean state (no cached `.olean` files) is the only reliable compilation test. Warm builds reuse stale objects and can mask broken imports, namespace collisions, dangling opens after refactoring, and cross-file definition drift. Single-file `lake env lean` passes are necessary but not sufficient: a file can compile alone while its downstream consumers break. Every merge into the main branch requires a cold build to pass.
3. **The non-vacuity check.** A clean-3 theorem can still be vacuous if its hypotheses are unsatisfiable or its conclusion is trivially true. The playbook requires a concrete *witness*: for a computability theorem, a numerical integration that matches the certified limit; for a universality theorem, a specific machine instance. Spec files for AI agents explicitly list *banned identifiers* to prevent the shortest-path escape of defining the target in terms of itself.
4. **The version-proliferation ban.** AI agents frequently produce versioned copies of the same file (`V3`, `V4`, `Final`, `Capstone`) rather than fixing the original. Each version may carry its own `sorry`s, polluting the import closure; a “clean” capstone that transitively imports a `sorry-carrying` predecessor is not clean. The playbook enforces: one theorem, one live file; when a new version lands, the old one is deleted or quarantined in the same commit.
5. **The counterexample-before-attack rule.** Before assigning an agent to prove a residual goal, the playbook requires an independent attempt to *disprove* it — concretely, by searching for counterexamples at degenerate and boundary configurations. A false residual (produced by over-generalization during reduction) can never be discharged; without this check, agents spend unbounded effort on an impossible target. Cross-verification by a second, independent model is the most reliable method.

No `native.decide` enters any headline construction. It survives only in auxiliary sites (a modular-form Sturm certificate and a compiled-example index check), on which no result above depends.

11 Conclusion

Ripple brings the molecular-programming theory of CRN-computable real numbers into a proof assistant for the first time, and carries it across the full ladder of models — the GPAC / CRN continuum, the large-population-protocol compilation pipeline, and the continuous-time Markov chain layer bridged to the deterministic limit by Kurtz’s mean-field theorem. The development is large: roughly 768k lines of Lean 4 across more than 900 files, much of it foundations Mathlib did not previously have — a verified CTMC and Kurtz mean-field theory, a Frobenius regular-singular analysis built from scratch, and complete formalizations of the BGP and SCWB Turing-completeness

theorems. The core constructions are verified axiom-free, and wherever a result is conditional or a published argument has a gap, the paper says so precisely.

Two things stand out. First, formalization did real mathematical work. It exposed genuine, fixable gaps in published proofs — the approximate-majority drift inequality, the LPP main theorem, and the Catalan-conjecture dependency hidden in an urn-model argument — and it proved new results, notably that Apéry’s constant $\zeta(3)$ is real-time CRN-computable through its holonomic generating function, while sharpening the modular $1/\pi$ question into a clean open problem. Second, essentially all of this Lean was written by AI agents running *publicly available* models (Claude Opus/Fable, GPT-5.x), with the human contribution limited to direction and review — evidence that AI-assisted formalization at this scale is now practical and reproducible. We offer Ripple as an open foundation for others to build on.

References

- [1] Marie Albenque and Lucas Gerin. On the algebraic numbers computable by some generalized Ehrenfest urns. *Discrete Mathematics & Theoretical Computer Science*, 14(2):271–284, 2012.
- [2] Dana Angluin, James Aspnes, and David Eisenstat. A simple population protocol for fast robust approximate majority. *Distributed Computing*, 21(2):87–102, 2008.
- [3] Roger Apéry. Irrationalité de $\zeta(2)$ et $\zeta(3)$. *Astérisque*, 61:11–13, 1979.
- [4] Frits Beukers. A note on the irrationality of $\zeta(2)$ and $\zeta(3)$. *Bulletin of the London Mathematical Society*, 11(3):268–272, 1979.
- [5] Olivier Bournez, Pierre Fraigniaud, and Xavier Koeqler. Computing with large populations using interactions. In *Proceedings of the 37th International Symposium on Mathematical Foundations of Computer Science (MFCS)*, pages 234–246. Springer, 2012.
- [6] Olivier Bournez, Daniel S. Graça, and Amaury Pouly. Polynomial time corresponds to solutions of polynomial ordinary differential equations of polynomial length. *Journal of the ACM*, 64(6):38:1–38:76, 2017.
- [7] Janna Burman, Ho-Lin Chen, Hsueh-Ping Chen, David Doty, Thomas Nowak, Eric Severson, and Chuan Xu. Time-optimal self-stabilizing leader election in population protocols. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 47–57. ACM, 2021.
- [8] J. W. S. Cassels. On the equation $a^x - b^y = 1$, II. *Mathematical Proceedings of the Cambridge Philosophical Society*, 56(2):97–103, 1960.
- [9] Ho-Lin Chen and Xiang Huang. Bounded analog complexity of real numbers. In *Proceedings of the 32nd International Conference on DNA Computing and Molecular Programming (DNA)*, LIPIcs, 2026. To appear.
- [10] David Doty, Mahsa Eftekhari, Leszek Gasieniec, Eric E. Severson, Przemyslaw Uznanski, and Grzegorz Stachowiak. A time and space optimal stable population protocol solving exact majority. In *Proceedings of the 62nd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1044–1055, 2021.
- [11] W. T. Gowers, Ben Green, Freddie Manners, and Terence Tao. On a conjecture of Marton. *Annals of Mathematics*, 201(2):515–549, 2025. Formalized in Lean 4 (the PFR project), December 2023.
- [12] Vera Hárs and János Tóth. On the inverse problem of reaction kinetics. In *Qualitative Theory of Differential Equations*, volume 30 of *Colloquia Mathematica Societatis János Bolyai*, pages 363–379. North-Holland, 1981.

- [13] Xiang Huang and Rachel N. Huls. Computing real numbers with large-population protocols having a continuum of equilibria. In *28th International Conference on DNA Computing and Molecular Programming (DNA 28)*, volume 238 of *LIPIcs*, pages 7:1–7:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022.
- [14] Xiang Huang, Titus H. Klinge, and James I. Lathrop. Real-time equivalence of chemical reaction networks and analog computers. In *International Conference on DNA Computing and Molecular Programming*, pages 37–53. Springer, 2019.
- [15] Haruki Kanaya, Ryota Eguchi, Taisho Sasada, Fukuhito Ooshita, and Michiko Inoue. Time- and space-optimal silent self-stabilizing exact majority in population protocols. In *Proceedings of the 27th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 270–286, 2025. arXiv:2503.17652.
- [16] Thomas G. Kurtz. Solutions of ordinary differential equations as limits of pure jump Markov processes. *Journal of Applied Probability*, 7(1):49–58, 1970.
- [17] Preda Mihăilescu. Primary cyclotomic units and a proof of Catalan’s conjecture. *Journal für die reine und angewandte Mathematik*, 572:167–195, 2004.
- [18] OpenAI. Planar point sets with many unit distances, 2026.
- [19] Srinivasa Ramanujan. Modular equations and approximations to π . *Quarterly Journal of Mathematics*, 45:350–372, 1914.
- [20] Peter Scholze. Liquid tensor experiment. *Experimental Mathematics*, 31(2):349–354, 2022. Formalized in Lean by J. Commelin and the Lean community, completed July 2022.
- [21] Atle Selberg and Sarvadaman Chowla. On Epstein’s zeta-function. *Journal für die reine und angewandte Mathematik*, 227:86–110, 1967.
- [22] Claude E. Shannon. Mathematical theory of the differential analyzer. *Journal of Mathematics and Physics*, 20(1-4):337–354, 1941.
- [23] David Soloveichik, Matthew Cook, Erik Winfree, and Jehoshua Bruck. Computation with finite stochastic chemical reaction networks. *Natural Computing*, 7(4):615–633, 2008.
- [24] George Tsoukalas et al. Advancing mathematics research with AI-driven formal proof search, 2026. arXiv:2605.22763.
- [25] Alfred van der Poorten. A proof that Euler missed: Apéry’s proof of the irrationality of $\zeta(3)$. *The Mathematical Intelligencer*, 1(4):195–203, 1979.